

Assignment - 2 - Adv DevOps

Q.1) Create a REST API with serverless framework.

Ans Creating a REST API with serverless framework is an efficient way to deploy applications that can scale automatically without managing servers.

- i) Serverless framework: A Powerful tool that deployment of services and serverless applications across various cloud providers such as AWS, Azure and Google Cloud.
- ii) Serverless architecture: The design module allows developers to build application without worrying about underlying infrastructure. Enabling focus on code and business logic.
- iii) REST API: Representational State Transfer is architecture style for designing network applications.

Steps

- 1) Install serverless framework
You start by installing serverless framework CLI globally using node package manager. This allows you to manage serverless application from your terminal.
- 2) Creating a node.js serverless project.
A directory is created for your project where a `serverless` will initialize a serverless service. This service will house all your lambda functions. Set up a template for AWS node.js microservices.
- 3) Project structure
The project creates essential files like `handler.js` and `serverless.yml`.
- 4) Create a REST API resource.

- 5) Deploy The Service
With the sbx deploy commands ^{Package} serverless framework uploads necessary resources to ^{AWS} your applications.
- 6) Testing the API :
Once deployed you can test REST API's with ^{tools} like curl or postman by making Post request.
- 7) Storing data in Dynamo DB : To store Submitted candidate data you integrate AWS Dynamo DB as Database.
- 8) Adding more functionalities : Adding functionalities like list all candidates by get candidates by ID.
- 9) AWS IAM Permissions
You need to ensure that Serverless framework is given tight permission.
- 10) Monitoring and maintenance
Serverless framework provides service info like deployed endpoints, API keys, log streams.

Q.2) Case study for Sonarqube

1) Profile Creation in Sonarqube:

Quality profiles in Sonarqube are essential configuration that define rules applied during code analysis. Each project has a quality profile for every supported language with default being 'Sonar way' profile. Comes built in all languages. Custom profiles can be created by copying or extending existing ones. Copying creates an independent profile to specific projects. You can activate or deactivate rules whenever you want.

2) Using Sonar^{cloud}qube to analyze Github code.
It is a cloud based quality and security analysis tool that integrates seamlessly with Github. A case study involves a development team using Sonarcloud to analyze their project code. By integrating it in Github action, the team automated pull request analysis. This provided real time feedback of code smells, bugs, Vulnerabilities etc.

3) Install sonarlint in your java intellij ~~idea~~ and analyze your java code.
In a case study developer installs sonarlint in IntelliJ IDE to analyze Java project. Sonarlint flags code smells, bugs, potential security, in real time as we write code. Ensures better code quality and ^{avoids} technical debt.

4) Analyze Python Project in Sonarqube.
A development team integrates Sonarqube to analyze their Python project. By setting Sonarqube locally they run static code analysis and identify code smells, bugs, security threats. This continuous feedback helps to improve quality of codebase over time.

5) Node.js project with Sonarqube.
By integrating Sonarqube with their CI pipeline they continuously assess code for smells, bugs, security threats. Real time analysis helps to improve code quality.

Q.3) Terraform "self-service" infrastructure model

→ ① Terraform modules serve for self-service infrastructure

· Create Terraform modules that codify the standards for deploying the common resource like VPC's EC2 instances and S3 buckets.

Example module for EC2 instance.

```
Variable "instance type" {  
    default = "t2.micro"  
}
```

```
resource "aws_instance" "example" {
```

```
    ami = "ami-12345678"
```

```
    instance_type = Var.instance_type
```

```
    tags = {
```

```
        Name = "Example_instance"
```

```
    }
```

```
ec2 module outputs.tf:
```

```
output "instance-id" {
```

```
    value = instance.example.id
```

```
}
```

② Terraform Cloud integration to automate infrastructure request process, can run on ticket approval, automating resource deployment.

③ Creating terraform modules for teams.

Define reusable modules for commonly requested ^{model}

① Networking (VPC, subnets) ② Compute

③ Storage

④ IAM roles.

By this teams can manage their own infrastructure while maintaining compliance with organizational standards.